PROJECT REPORT

CG1111A

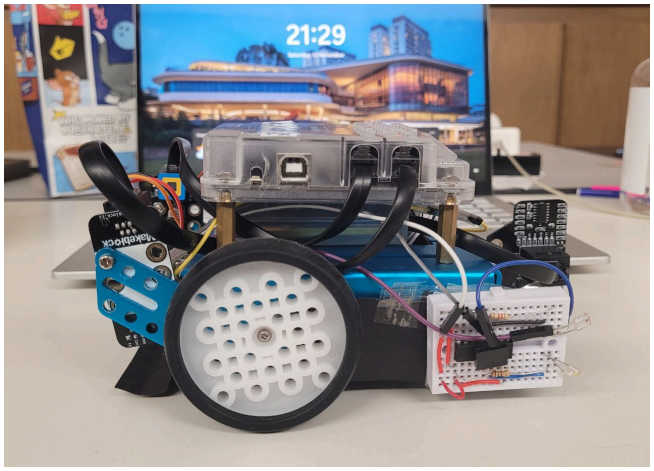
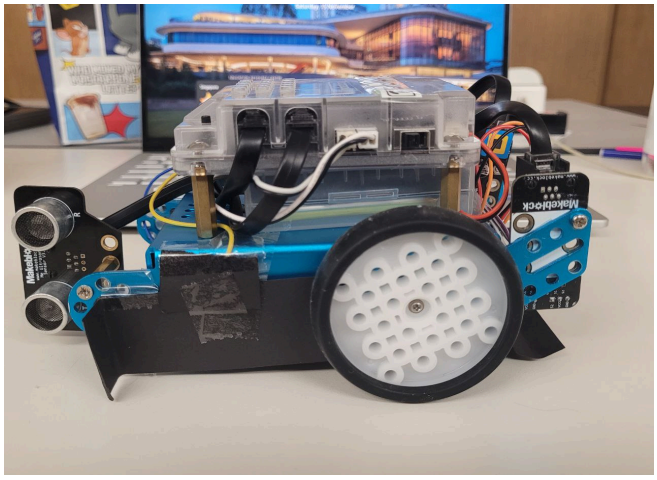
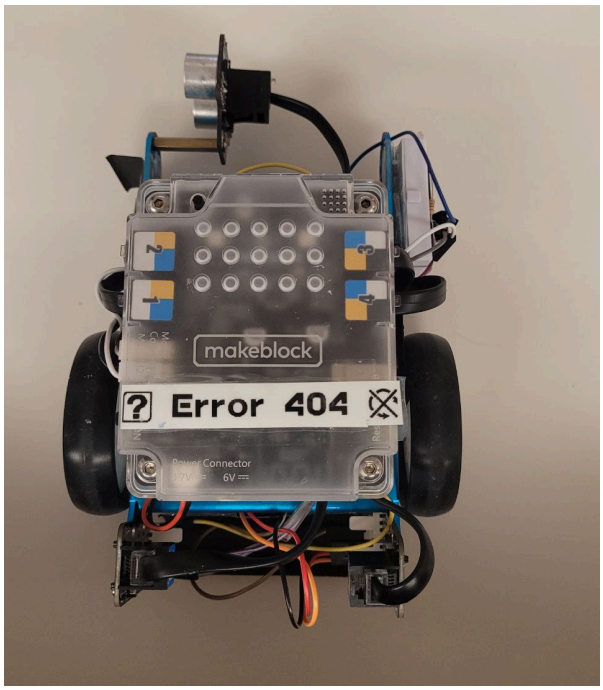
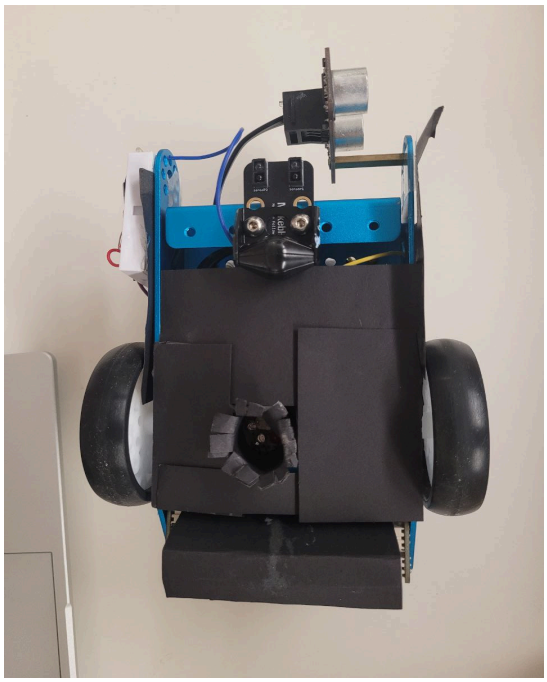
Group Name

Group Member	Student Number

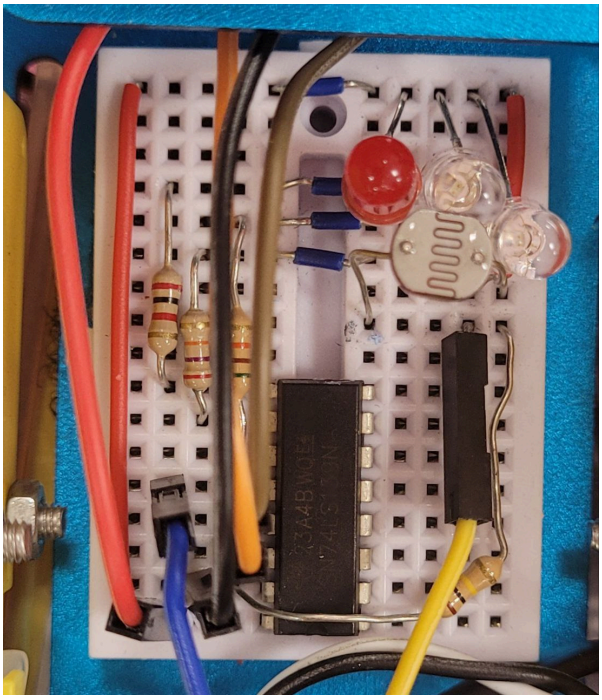
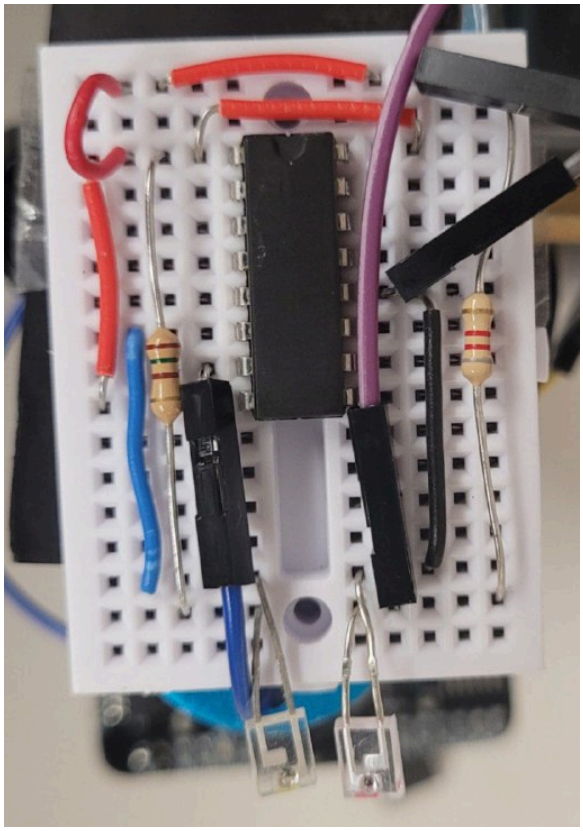
Table Of Contents

1. Pictures

1.1 Pictures of mBot

Right View	Left View
	
Top View	**Bottom View**
	

1.2 Pictures Of Circuits

LDR Circuit	IR Circuit
 **the chimney has been removed in order for circuit to be seen more clearly*	

2. Overall Algorithm

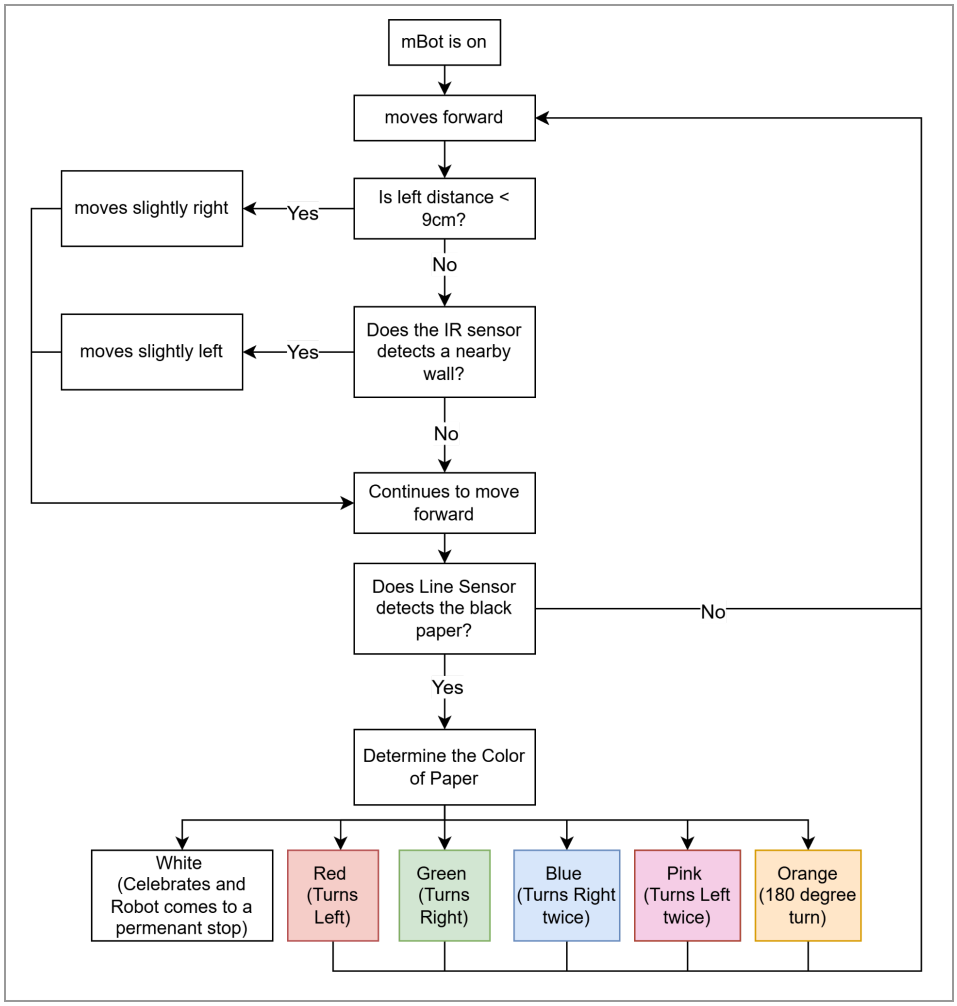


Fig 2: Flowchart of our overall algorithm i.e loop() function, in robot_project.ino

The objective of this project is to navigate our mBot through a maze without bumping into the walls whilst completing “challenges” determined by the color papers on the floor.

Upon activation, the mBot moves forward and continuously measures its distance from the left wall.

If the distance sensor indicates the robot is too close to the left wall, the system increases the speed of the left motor and decreases the speed of the right motor. Since the left wheel is spinning faster than the right, the robot pivots slightly to the right, moving it away from the left wall.

Conversely, if the robot detects it is too close to the right wall it decreases the speed of the left motor and increases the speed of the right motor. The faster right wheel causes the robot to pivot slightly left, moving it away from the right wall.

Otherwise, the robot continues to drive straight ahead.

When the line finder detects the black line, it uses the red, green, and blue LEDs sequentially to read the colour of the paper.

Depending on the colour read, the robot executes a specific task associated with that colour. Once the task (the "challenge") is complete, the robot updates its state for continued operation.

Notably, if the colour sensor detects white, it sounds the buzzer and comes to a permanent stop. If the detected colour is not white (i.e., it completes a challenge), the robot continues navigating until the next challenge.

File Organisation

By taking advantage of how the Arduino IDE automatically compiles all files within the same folder into a single project, we organised our code by separating related functions into dedicated files. This helps keep the overall project clean, structured, and easy to maintain. The files are as follows:

File Name	Purpose
robot_project.ino	The main program contains the setup() and loop() functions as well as call functions from other files.
0_headers.h	Contains the bulk of definitions used
A_Colors.h	Contains color matching algorithm as well as the code for calibrating said color matching algorithm (disabled for our final run)
B_Movement.h	Contains all the functions that dictate the Mbot's motions as well as the correction function to prevent it from colliding with the maze walls
C_Sensors.h	Contains the code for the linesensor as well as the celebrate() function which activates upon completion of the maze.

3. Subsystem Algorithms

3.1 Colour Sensors

To complete the "challenges", we built an Light-Dependent Resistor (LDR) based color sensor that measures the reflected intensities of red, green, and blue light and identifies the object's colour by comparing the readings to the characteristic RGB components of light reflected by specific colored surfaces, such as red, orange, pink, green, blue, and white paper.

Due to the limited number of available pins, we used a 2-to-4 Decoder IC Chip to control the 3 LEDs.

- When S1 and S2 are HIGH, Red LED turns on
- When S1 is HIGH and S2 is LOW, Green LED turns on
- When S1 is LOW and S2 is HIGH, Blue LED turns on

At fixed intervals, we toggled the state (HIGH or LOW) of the two pins to switch on each LED. We then measured the voltage divider reading of the LDR through A2.

Using the function **setBalance()** to calibrate the color sensor, we measured the RGB values reflected by white and black colored papers on the field. The white measurement represents the readings when all light is reflected, while the black measurement represents the readings when most light is absorbed. The greyscale is then determined by subtracting the black values from the white values.

When we reached a colored paper and recorded the LDR readings into “ledArray”, we utilise the following formula to scale it close to RGB range (0-255):

(ledArray - blackArray / greyDiff) *255

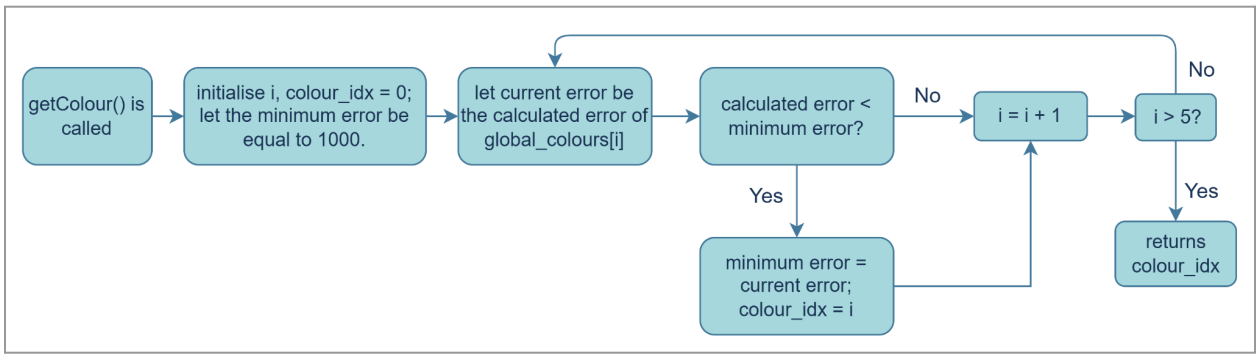


Fig 3.1: Flowchart of **getColour()** function in Colors.h

Next, we utilize the **get_Colour()** function which takes the measured color and compares it against a list of known colors stored in the “global_colours array”.

1. It starts by assuming the first color in the reference list is the best match.
2. It then goes through every other reference color, one by one.
3. In each step, it calls **get_error()** to find the distance between the colors .
4. If the new error is smaller than the current smallest error (min_error), it updates the stored minimum error and saves the index (the position) of that best-matching color.

Once it checks all the reference colors, it returns the index of the color that was the closest match (the one with the minimum error). It also calls the **show_color()** function to display the identified color.

3.2 Navigation

Depending on the detected colour as follows, our robot would take the following motions:

Colour	Motion	Functions
Red	Left-Turn	turnLeft()
Green	Right-Turn	turnRight()
Orange	180° turn within the same grid	uTurn()
Pink	Two successive left turns	doubleLeftTurn()
Blue	Two successive right turns	doubleRightTurn()

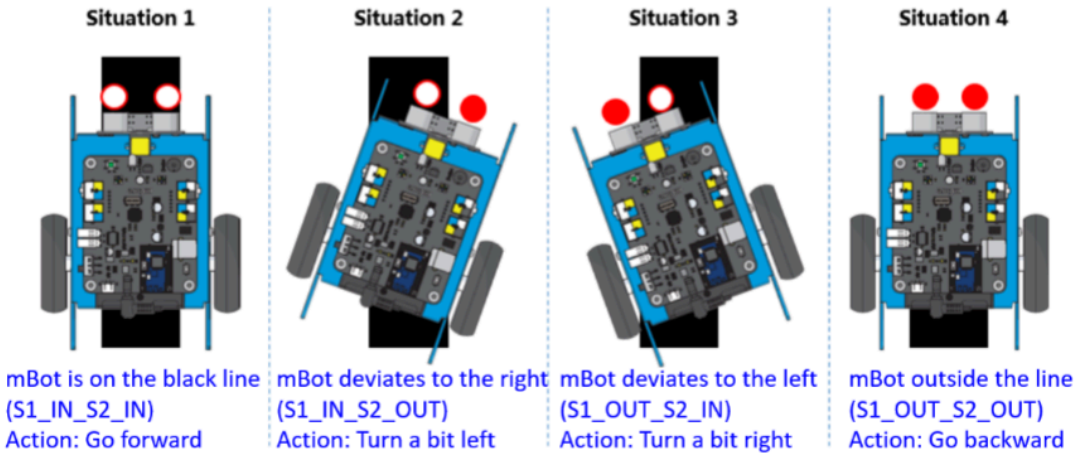
3.3 Line Sensor

```
bool reached_paper() {
  int sensorState = lineFinder.readSensors();  // read the line sensor's state

  if (sensorState == 3) {
    return false;
  }
  return true;
}
```

Code 3.3: Code for function **reached_paper()** located in C_Sensors.h which utilises the mBot's line sensor to determine whether there is a colored paper underneath it.

There are 4 possible states that the lineFinder.readSensors() returns:



If sensorState = 3, it means the mBot has not reached the colored paper (Situation 4) and hence the Mbot will continue moving forward. Elsewise, the mBot lies at least partially on the black line and thus stops to detect the color of the colored paper.

3.4 IR Sensor & Ultrasonic Sensor

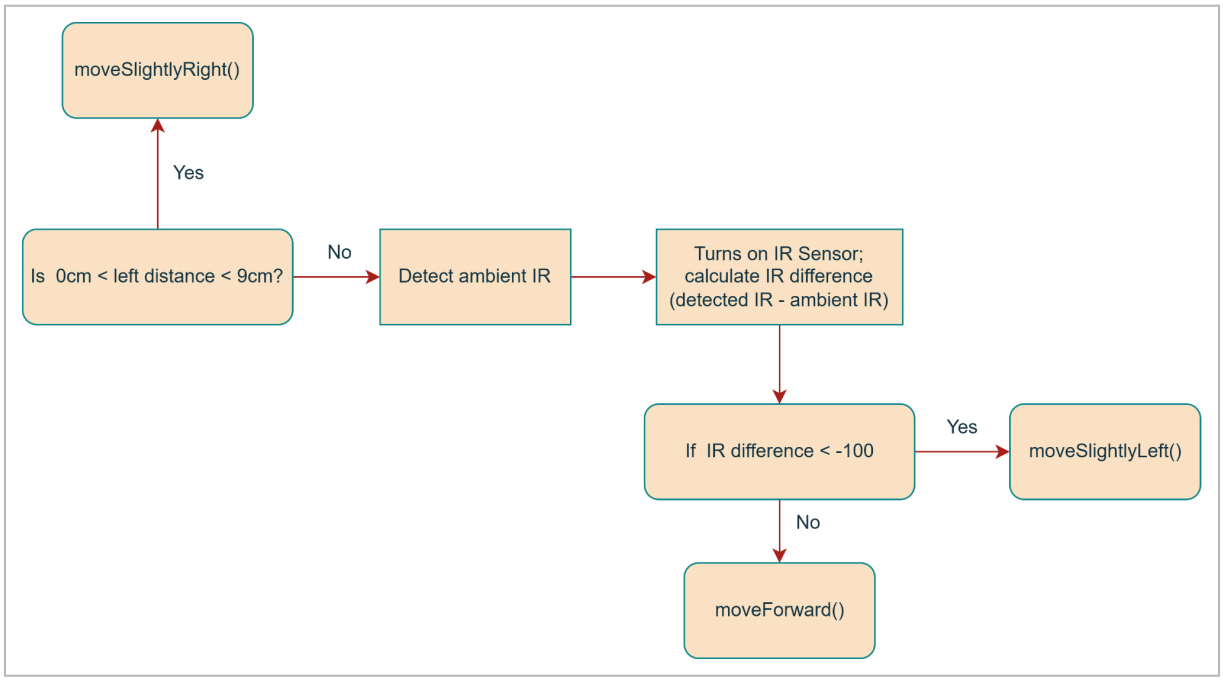


Fig 3.4: Flowchart of **correctionFunction()** in B_Movements.h

To ensure the robot moves straight and avoids collisions within the maze, we used both an ultrasonic sensor and an IR detector.

- The ultrasonic sensor measures the distance to the wall on the left. If this distance drops below a safety threshold, the robot steers slightly right.
- The IR detector measures the distance to the wall on the right. If this distance drops below its threshold, the robot steers slightly left.

The ultrasonic sensor emits a pulse and calculates the distance from the wall using the travel time for the echo to return as well as the known speed of sound.

To ensure the IR detector provides stable readings unaffected by light variations, the system first detects ambient light reading when the IR emitter is turned off. Afterwards, it detects the light reading with the IR emitter on.(i.e S1 and S2 are both LOW). The difference between the two reading values provided is then used to determine proximity to the right wall.

Should either ultrasonic sensor or IR detector determine that the mBot is too close to the wall, they would call upon **moveSlightlyRight()** and **moveSlightlyLeft()** respectively. This is achieved by adding/subtracting a pre-defined value called CORRECTION_SPEED (equals to 35). For instance, to move the mBot slightly left by increasing the left motor speed and decreasing the right motor speed and thus add CORRECTION_SPEED to both motors.

```
void moveSlightlyLeft() {
    leftMotor.run(motorSpeed + CORRECTION_SPEED);
    rightMotor.run(-motorSpeed + CORRECTION_SPEED);
}
```

Code 3.4: Code for function **moveSlightlyLeft()** in B_Movements.h

4. Calibrations/Improvements

4.1 Color Sensor Calibration

These are the steps taken to calibrate the robot to detect the 6 colours (red, blue, green, orange, pink, white).

1. First, we use the **set_balance()** function to set the values of blackArray, whiteArray and greyDiff. As previously mentioned, these are the values we would use to scale the detected colour to a value between 0 and 255.

2. Next, we start testing the robot on the different colours. We test each colour multiple times, then take the average R, G & B values of these colours, before hardcoding these values into an array of colours which we would use for further runs.

3. For colour detection in future runs, we would use an error function that calculates the euclidean distance between 2 points to determine which colour falls closest to our detected colour. Each colour can be thought of as a 3D point with its R, G and B values. For instance, when the robot detects a colour and outputs the R, G and B values, we compare these values with each of the different colour values we calibrated earlier by calculating the euclidean distance. The calibrated colour with the lowest euclidean distance calculated will be the closest match, so we would take that calibrated colour as the result.

While this would work in theory, in reality there are many other external factors that cause the colour detection to be very inconsistent.

The main factor would be the sensitivity of our LDR colour sensor. Our colour sensor is extremely sensitive, so any slight change in the ambient light would cause the RGB outputs for the various colours to change drastically.

Although the Euclidean distance method is theoretically sound, the high sensitivity of our LDR color sensor leads to significant inconsistencies. Even minor changes in ambient light drastically shift the RGB outputs (e.g., Green shifting from R:100, G:180, B:110 to R:80, G:140, B:70), which in turn changes the calculated Euclidean distances and causes detection failures.

To improve robustness, we implemented physical countermeasures:

- We constructed a chimney around the three LEDs and the LDR to isolate the sensor.
- We applied black paper skirting around the robot's base to minimize ambient light entering the bottom of the mBot.

These modifications ensure a more stable sensing environment for the LDR, resulting in more consistent and reliable color detection values.

4.2 IR Sensor

Based on circuit testing, the IR detector's accuracy is reliable up to approximately 6cm. However, due to inherent inconsistency—likely caused by fluctuating ambient light—we limit the detector's operational role to critical, extremely close-range detection (less than 3cm) for collision avoidance.

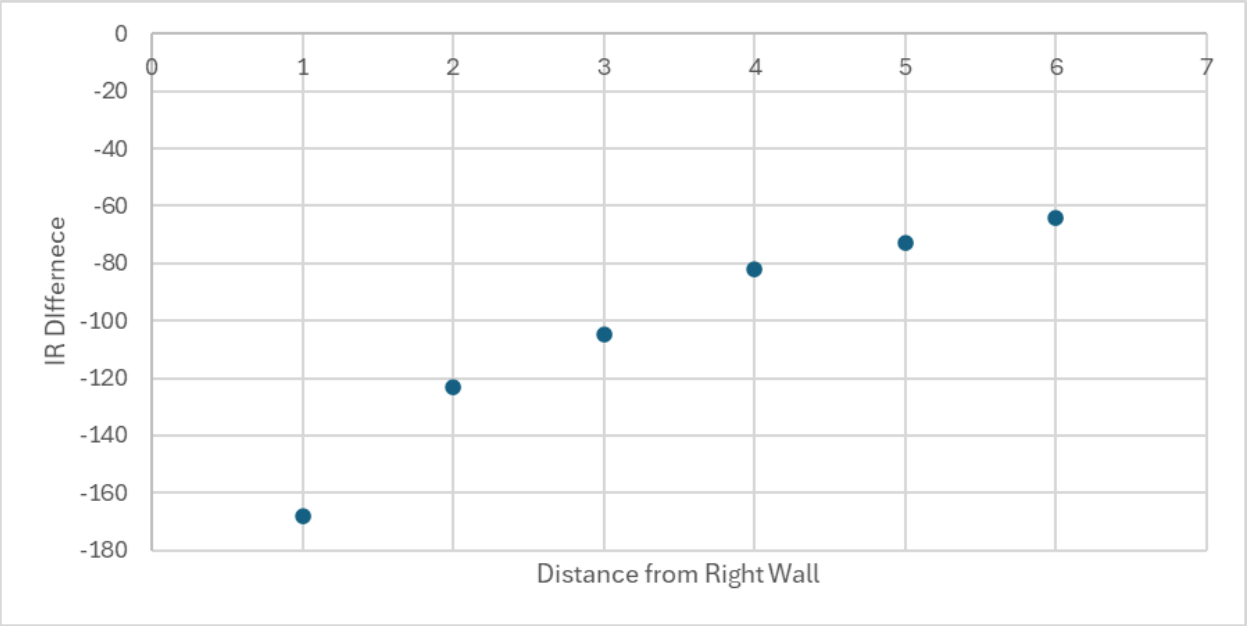


Fig 4.2: Graph showing relationship between recorded IR Difference(V) and distance from wall(cm) during our initial testing of IR detector.

After several test runs and adjustments, we determined that the mBot should turn slightly left whenever its IR difference falls below –100, as this threshold consistently prevents it from colliding with the maze walls.

5. Hardware

A. Platform used - mBot Robotic Platform

This platform comes with two pre-made sensors - a Line sensor and an Ultrasonic Sensor. Additionally, we included two custom-built sensors, namely the LDR colour sensor and the IR Proximity sensor, the workings and purpose of which will be elaborated on hereafter.

B. Light Dependent Resistor (LDR) Colour Sensor

The colour sensor is built from the following circuit components:

- An LDR (Light Dependent Resistor)
- Three LEDs (Red, Green, Blue)
- A voltage divider system to read the LDR voltage via the mCore's analog pin
- A 2-to-4 decoder (HD74LS139P) to turn LEDs on selectively

Connection to mCore : The LDR sensing circuit is interfaced with the mCore using Port 4, as this port provides the necessary analog input pins (A0–A3). An RJ25 adapter breaks the port into VCC (5V), GND, and an analog signal pin, which are routed to the breadboard.

LDR Voltage Divider : The LDR is connected in series with a fixed resistor (RLDR) on the 170-point mini breadboard to form a voltage divider. The midpoint of this divider is connected to the analog signal pin from the RJ25 adapter. This allows the mCore to measure voltage changes caused by varying light intensity.

LED Illumination System : Three LEDs (red, green, blue) are mounted on the same breadboard and oriented downward to illuminate the colour paper. These LEDs are controlled using the HD74LS139P 2-to-4 decoder, which enables the mCore to activate each LED individually using only two digital signal pins. Each LED includes a current-limiting resistor to keep current within safe limits.

Physical Mounting and Shielding : All components are wired using short, single-core jumper wires to ensure tight and reliable connections. The mini breadboard is attached beneath the mBot so the LEDs shine directly on the maze surface and the LDR receives the reflected light. A black-paper skirting is added around the module to block ambient light and ensure stable and consistent readings.

Operating Principle

The colour sensor works based on reflective colour sensing:

1. You shine one LED at a time (red, green, blue) onto the coloured paper.
2. The paper reflects different amounts of each colour of light depending on its colour.
3. The LDR detects how bright the reflected light is:

   High brightness → LDR resistance decreases → LDR voltage drops
   Low brightness → LDR resistance increases → LDR voltage rises

4. The mCore reads this voltage using an analog pin

5. By comparing the three readings (R, G, B), your code determines which colour paper is underneath.

This colour sensor works because different coloured surfaces reflect and absorb light differently at different wavelengths. The three LEDs used emit light in distinct, narrow wavelength bands:

Red LED: ~620–630 nm; Green LED: ~520–530 nm; Blue LED: ~460–470 nm

When each LED shines onto the coloured paper, the amount of light reflected back to the LDR depends on how the material interacts with that specific wavelength.

Note - A coloured surface looks a certain colour because of its spectral reflectance profile: Red paper reflects strongly around 620–700 nm, but absorbs shorter wavelengths (green/blue). Green paper reflects strongly around 520–560 nm, but absorbs red and blue. Blue paper reflects strongly around 450–480 nm, but absorbs red and green. Mixed colours (orange, pink, etc) reflect in specific combinations of these regions.

This means that when shining a red LED on red paper, most of the light is reflected, but when shining a blue LED on red paper, most of the blue light is absorbed, resulting in a much lower reflected intensity. The LDR is not colour-sensitive. It only detects brightness, that is, the total light intensity reaching it. Since only one wavelength band is shone onto the paper at a time, the brightness it detects is directly tied to how much of that specific wavelength the surface reflects.

LDR Behavior

More reflected light → higher intensity → LDR resistance ↓ → Voltage ↓

Less reflected light → lower intensity → LDR resistance ↑ → Voltage ↑

By reading the voltage from the LDR's voltage divider:

$$V = \frac{R(LDR)}{R(LDR) + R} \times 5V$$

A distinct value for each LED and each type of paper can be obtained. The reading for each of the three LEDs are then combined and used to infer the colour of the paper.

For example, for Red paper:

Strong reflectance at ~630 nm → Low LDR voltage under Red LED; Absorbs green/blue → Higher LDR voltage under Green & Blue LEDs

The code then compares these values to calibrated thresholds to determine which colour is being detected.

LED Control Using the 2-to-4 Decoder

To control the three colour LEDs used for colour sensing, a way to individually switch on the red, green, and blue LEDs is needed, (while keeping in mind that we must use as few mCore pins as possible). The

mCore only provides four available signal pins for all custom sensors, but the RGB LEDs alone would require three separate digital outputs, and the IR emitter requires another.
To overcome this limitation, we use a 2-to-4 decoder, which allows us to select one of four outputs using only two input pins from the mCore. This ensures that only one LED is turned on at any given time while keeping the design pin-efficient and leaving the remaining pins free for the LDR and IR detector's readings.

The pin mappings for the three LEDs are as follows -

Y_1 - Blue LED
Y_2 - Green LED
Y_3 - Red LED

The Analog pin used to read the voltage value is A_2.

C. Infrared Sensor (IR)

Infra-Red Proximity Sensor
The IR sensor is built from the following circuit components:

- An IR Emitter
- An IR Sensor
- A voltage divider system to read the IR detector voltage via the mCore's analog pin (A3)
- A L293D Motor Driver to turn on and off the IR emitter via the pin Y0 (from 2-to-4 decoder (HD74LS139P))

Connection to mCore: The IR sensor is connected to the mCore using Port 4. The control signal for enabling and disabling the sensor is controlled using the output signal Y0 from the LDR. This control signal is routed to a L293D motor driver to amplify the current needed for the IR when toggled on. When the IR sensor is toggled on, the voltage corresponding to the reflected IR level is fed back to the mCore through analog pin 3 (A3) for processing.

Physical Mounting: Since the IR sensor is inaccurate over long distances, we placed the IR sensor to the front-right of the mBot to minimize error and reduce noise from other factors. This allowed for stable and accurate readings when detecting a wall.

Operating Principle
The IR sensor operates by comparing the infrared light reflected from the environment when the IR emitter is off against when it is turned on. The process works as following:

1. Measure ambient IR
   First, ensure that the IR emitter is kept off, ensuring the control pins S1 and S2 and not both LOW (for our case, we use S1 HIGH and S2 LOW). This allows us to read the ambient IR, thus accounting for the background environment

2. Turn on the emitter

The IR emitter is then turned on by setting S1 and S2 to LOW, enabling the decoded output (Y0) and activating the L293D motor driver. This provides the necessary current to the IR emitter, and thus projecting additional IR into the environment.

3. Reading the reflected IR value
   If an object (in our case, maze wall) is present, some of the IR light emitted will be reflected back towards the receiver. The amount reflected back depends on several physical factors such as distance and surface colour.

   The IR receiver is a photoconductor whose resistance changes based on the IR light falling on it. We detect the voltage value just before the receiver, that is, create a voltage divider using a fixed resistance and the IR receiver, and take the voltage reading at the midpoint.

More reflected IR → receiver conductance ↑ → effective resistance ↓ → Voltage reading at A3 ↓

Less reflected IR → receiver conductance ↓ → effective resistance ↑ → Voltage reading at A3 ↑

4. Determine proximity
   Compare the detected IR values with the ambient IR values recorded earlier. If there is a huge drop/difference between the two, it implies that an object is present. If there is only a small difference, then either no object is present or it is far away.

   In our case, if our (detected IR - ambient IR) < -100, we adjust course and move left slightly.

6. Work Divisions

Member's Name	Roles
Smeet	1. IR Sensor Assembly 2. Contributed to the Report
Cheuk Yin	1. Arduino Code 2. LDR Color Sensor Assembly 3. Contributed to the Report
Lawrence	1. Arduino Code 2. LDR Color Sensor Assembly 3. Contributed to the Report
Anannya	1. Robot Assembly 2. LDR Color Sensor Assembly 3. Contributed to the Report

7. Challenges Faced

Component	Description of Problem	Devised Solution
Wheels	● The right wheel was faulty and did not work, causing the Mbot to turn right when instructed to move forward.	● We tried changing the motor, but it still occasionally wouldn't turn. ● Realized this is due to the tightness of the wheel's screw and the unevenness of the skirting. ● Loosened the screw and trimmed the skirting.
LDR Sensor	● Changed readings depending on ambient lighting as miniscule as people walking around the table.	● Experimented with different resistance values to correctly calibrate sensitivity. ● Added skirting using black paper all around the MBot's base. ● Enclosed the three LEDs and the sensor in a "chimney" crafted from black paper to further block out light.
Wiring	● Wires and cables kept bumping into walls. ● Wires are messy and loose; causing the color sensor to not work	● Used shorter wires and resistors ● Taped wires together as a bundle for easier debugging and to ensure no protruding wires
IR Sensor	● Did not show much voltage variation in the output values.	● Varied resistance through trial and error to get the desired output.
Line Sensor/Code	● Did not stop initially even though it detected a black piece of paper.	● Realised we added a delay in our move forward function, causing each iteration of our loop to last much longer. ● As a result, the function that detects whether we hit a black paper or not only runs once every second, causing the sensor to miss the paper. ● To fix this, we removed the delay in our move forward function.